- Which mutexes might other threads hold?
- Are there any ordering requirements between the operations done in this thread and those done in another? How are those requirements enforced?
- Is the data loaded by this thread still valid? Could it have been modified by other threads?
- If you assume that another thread could be modifying the data, what would that mean and how could you ensure that this never happens?

This last question is my favorite, because it really makes me think about the relationships between the threads. By assuming the existence of a bug related to a particular line of code, you can then act as a detective and track down the cause. In order to convince yourself that there's no bug, you have to consider every corner case and possible ordering. This is particularly useful where the data is protected by more than one mutex over its lifetime, such as with the thread-safe queue from chapter 6 where we had separate mutexes for the head and tail of the queue: in order to be sure that an access is safe while holding one mutex, you have to be certain that a thread holding the *other* mutex can't also access the same element. It also makes it obvious that public data, or data for which other code can readily obtain a pointer or reference, has to come under particular scrutiny.

The penultimate question in the list is also important, because it addresses what's an easy mistake to make: if you release and then reacquire a mutex, you must assume that other threads may have modified the shared data. Although this is obvious, if the mutex locks aren't immediately visible—perhaps because they're internal to an object—you may unwittingly be doing exactly that. In chapter 6 you saw how this can lead to race conditions and bugs where the functions provided on a thread-safe data structure are too fine-grained. Whereas for a non-thread-safe stack it makes sense to have separate `top()` and `pop()` operations, for a stack that may be accessed by multiple threads concurrently, this is no longer the case because the lock on the internal mutex is released between the two calls, and so another thread can modify the stack. As you saw in chapter 6, the solution is to combine the two operations so they are both performed under the protection of the same mutex lock, thus eliminating the potential race condition.